Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра **гуманитарных, математических и естественно научных дисциплин**

ПРАКТИЧЕСКАЯ РАБОТА

Дисциплина(модуль)/предмет **МДК.03.01**

Тема: **Практическая работа №8. Сводный отчёт (ПК №19–№24)**

ВАРИАНТ № **3**

Студент (ка)/учащийся: **Браунков Матвей Константинович** / _____ /
Ф. И. О. подпись

Группа / Класс И14-23-9

Руководитель: **Чудинова А.А.** _____ /_____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «____» _____ 202__г.

Работа аттестована с оценкой _____
прописью

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3
1 ПК №19. АУДИТ ЗАВИСИМОСТЕЙ ПРОЕКТА ЧЕРЕЗ NUGET API.....4
2 ПК №20. МОНИТОРИНГ ВНЕШНЕГО API И СТРУКТУРИРОВАННЫЙ ЛОГ SERILOG.....9
3 ПК №21. АРМ БИБЛИОТЕКИ: ИНСПЕКТОР ЗАВИСИМОСТЕЙ И ПРОТОКОЛ SAT.....13
4 ПК №22. СИСТЕМА АВТОМАТИЧЕСКОГО ОБНОВЛЕНИЯ (AUTO-UPDATER).....16
5 ПК №23. АВТОНОМНЫЙ ТЕРМИНАЛ СТОЛОВОЙ (EF CORE SQLITE И POLLY).....21
6 ПК №24. ИНФОРМАЦИОННАЯ БЕЗОПАСНОСТЬ АРМ: BCRYPT, X.509 И АУДИТ SERILOG.....25
ЗАКЛЮЧЕНИЕ.....30
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....31

ВВЕДЕНИЕ

Цель работы: свести в один отчёт результаты шести промежуточных контролей по теме 1.1 «Основные методы внедрения и анализа функционирования ПО» (№19–№24) и закрепить навыки автоматизации внедрения, аудита, журналирования, обновления и защиты клиентских модулей АРМ ИС «Умный кампус» на C# (.NET 8).

Вариант 3 во всех работах. Каждая работа выполнена как отдельная программа, результаты сведены в главы. Общая схема сопровождения: проверка зависимостей (№19, №20), приёмка и протокол SAT (№21), автоматическое обновление (№22), автономная работа и синхронизация (№23), безопасность и аудит (№24).

Таблица 1 – Состав сводного отчёта

№	**Работа**	**Ключевые технологии**
19	Аудит зависимостей через NuGet API	HttpClient, NuGet API, Markdown
20	Мониторинг доступности API и лог Serilog	Serilog, Stopwatch, NuGet
21	АРМ библиотеки: дымовые тесты и протокол SAT	Переменные окружения, SHA-256, Serilog
22	Auto-Updater: обновление, SHA-256, откат	SemVer, manifest.json, UpdaterHelper
23	Автономный терминал столовой	EF Core SQLite, Polly, Outbox
24	Безопасность АРМ: BCrypt, X.509, аудит	BCrypt, X509Certificate2, Serilog

1 ПК №19. АУДИТ ЗАВИСИМОСТЕЙ ПРОЕКТА ЧЕРЕЗ NUGET API

Цель работы: закрепить навыки автоматизации проверки окружения и формирования эксплуатационной документации на C# (.NET 8): написать программу, которая через публичный NuGet API узнаёт актуальные версии пакетов, сравнивает их с локальными и формирует отчёт для администратора в формате Markdown (подход Docs-as-Code).

1.1 Описание задания

Вариант 3: «Аудит зависимостей проекта через NuGet API». Приложение обращается к официальному публичному API https://api.nuget.org/v3/index.json [1], проверяет актуальную версию популярных пакетов (Serilog и Newtonsoft.Json), сравнивает её с локальной версией и формирует отчёт для администратора.

Требования методических указаний: проект Console App (.NET 8), запросы через стандартный класс HttpClient [2], обработка сетевых ошибок (try-catch и таймаут), результат сохраняется в текстовый файл .md.

Таблица 1 – Схема работы программы

Шаг	**Действие**
1	Запросить индекс NuGet и найти ресурс PackageBaseAddress [3]
2	Для каждого пакета получить список всех версий
3	Отбросить предварительные версии (beta, rc) и выбрать максимальную
4	Сравнить с локальной версией и определить статус
5	Записать отчёт nuget_audit_report.md

1.2 Программный код с комментариями

Список локальных пакетов хранится в словаре. Таймаут HttpClient равен 10 секундам, поэтому программа не зависнет при плохой сети. Для каждого пакета определяется статус «Актуальна», «Устарела, нужно обновить» или «Пакет не найден».

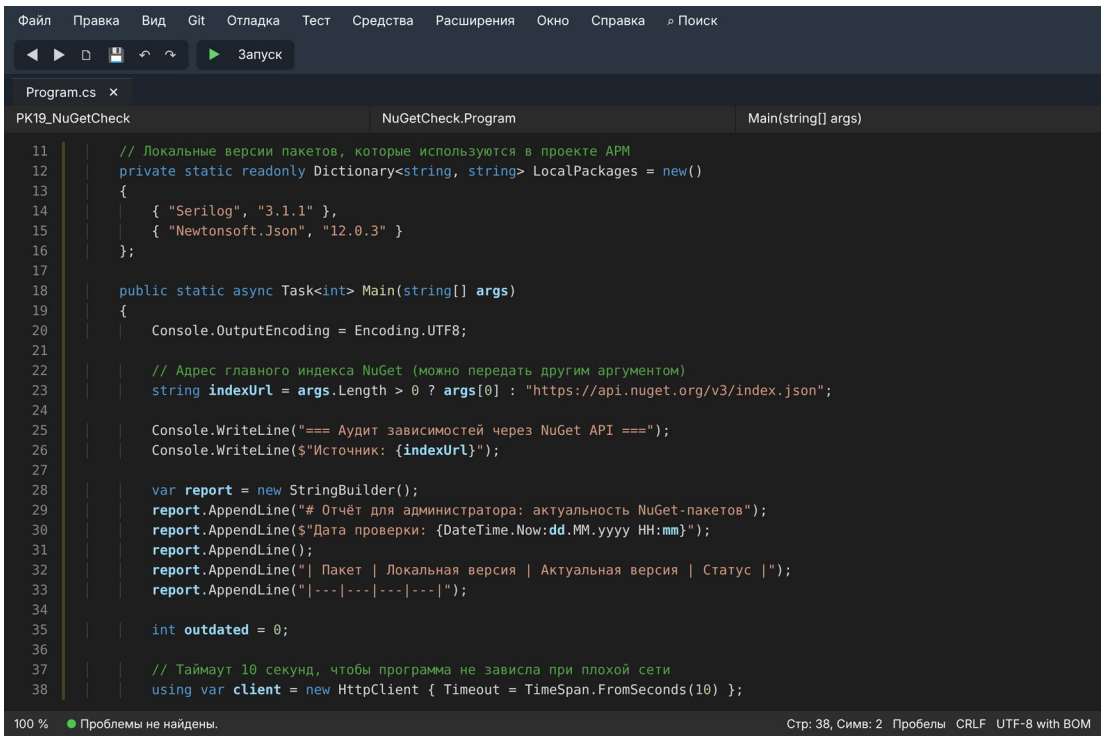


Рисунок 1 – Локальные версии пакетов, настройка HttpClient и главный цикл проверки

Ошибки сети и таймаут перехватываются в блоках catch, программа сообщает о проблеме и завершается с кодом 2. Отчёт собирается в StringBuilder и сохраняется в файл.

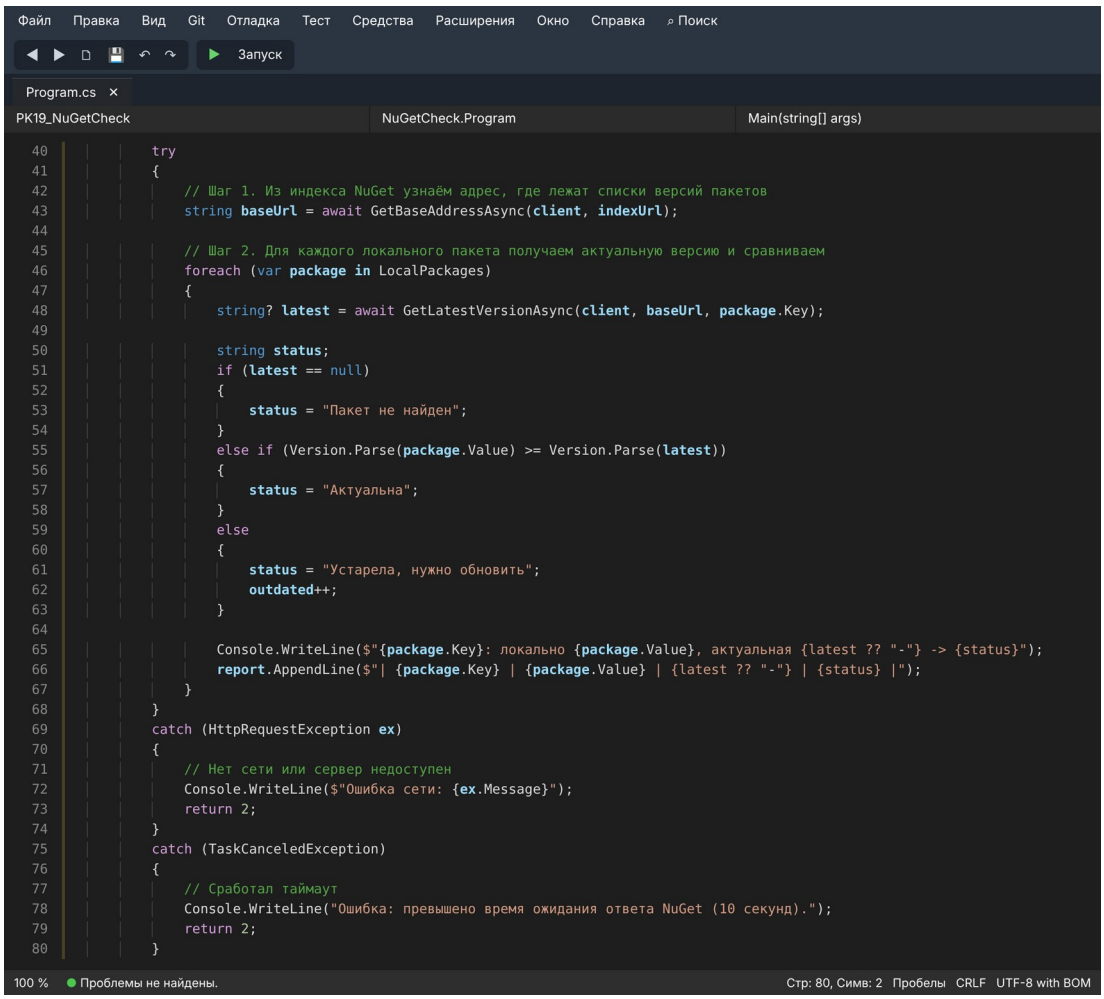


Рисунок 2 – Обработка сетевых ошибок и формирование Markdown-отчёта

Метод GetBaseAddressAsync берёт из индекса адрес, по которому лежат списки версий. Метод GetLatestVersionAsync пропускает версии с дефисом (beta, rc) и выбирает максимальную стабильную.

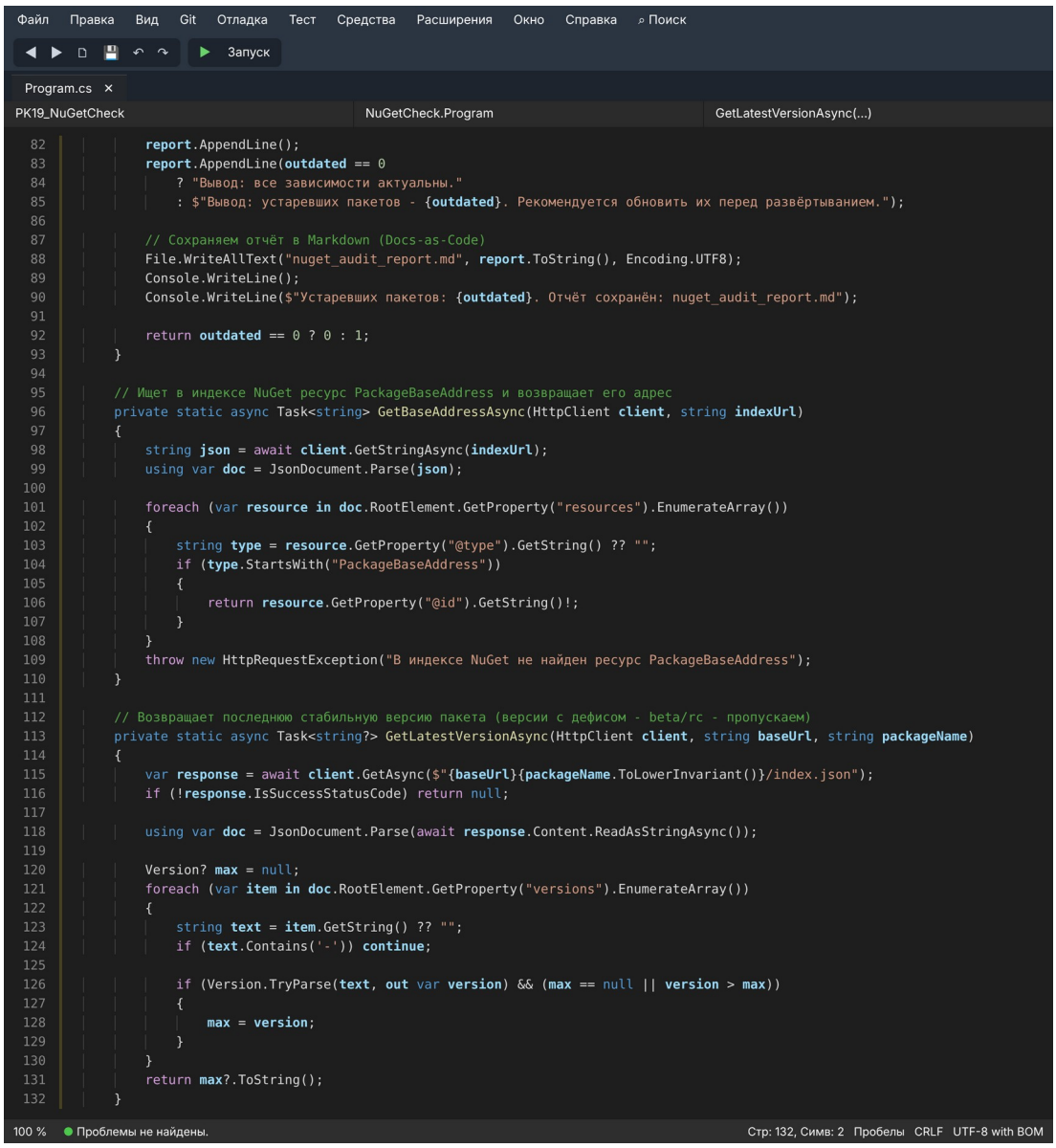


Рисунок 3 – Получение адреса PackageBaseAddress и последней стабильной версии

1.3 Результат работы программы

Программа возвращает код 0, если все пакеты актуальны, 1 – если найдены устаревшие, 2 – при сетевой ошибке. Это позволяет использовать её в скриптах развёртывания. Результат запуска показан на рисунке 4, а содержимое отчёта nuget_audit_report.md имеет вид таблицы из четырёх столбцов: пакет, локальная версия, актуальная версия, статус.

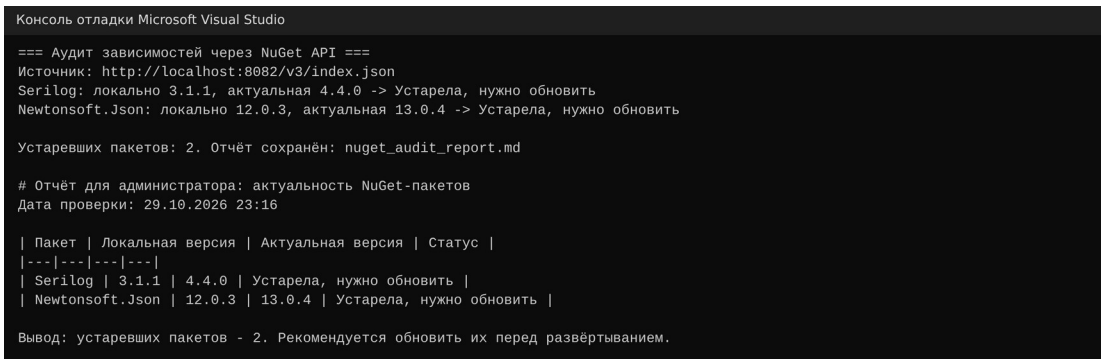


Рисунок 4 – Результат работы программы (Консоль отладки)

1.4 Вывод

В ходе работы разработана консольная программа на C# (.NET 8), которая проверяет актуальность NuGet-зависимостей через публичный API и формирует Markdown-отчёт для администратора. Программа обрабатывает сетевые ошибки и таймаут, не учитывает предварительные версии и возвращает код завершения, удобный для автоматизации.

Практическая польза: перед развёртыванием АРМ администратор быстро узнаёт, какие библиотеки устарели и могут содержать уязвимости.

2 ПК №20. МОНИТОРИНГ ВНЕШНЕГО API И СТРУКТУРИРОВАННЫЙ ЛОГ SERILOG

Цель работы: разработать консольное приложение для АРМ дежурного инженера, которое проверяет актуальность NuGet-пакета, замеряет время отклика (Latency) и записывает события через Serilog в консоль и локальный файл.

2.1 Описание задания

Вариант 3: «Проверка актуальности NuGet-пакетов». Программа проверяет зависимость проекта через официальный API пакетов https://api.nuget.org/v3/index.json [1], выявляет устаревшую библиотеку и формирует предупреждение. Как и в эталонном примере, фиксируется время отклика сервера, а события пишутся в журнал Serilog [4].

Логирование настроено на два приёмника: консоль и файл logs/arm-.log с ежедневным созданием нового файла (RollingInterval.Day) [5]. Уровни сообщений: Information – штатная работа, Warning – устаревший пакет или медленный ответ, Error – сервер недоступен.

Таблица 1 – Уровни логирования в программе

Событие	**Уровень**	**Код завершения**
Пакет актуален, ответ быстрый	Information	0
Пакет устарел или ответ дольше 1000 мс	Warning	1
NuGet недоступен или пакет не найден	Error	2

2.2 Программный код с комментариями

Stopwatch измеряет время ответа индекса NuGet. Параметры сообщений ({Ms}, {Package}) записываются в лог как структурированные поля, а не как склеенная строка.

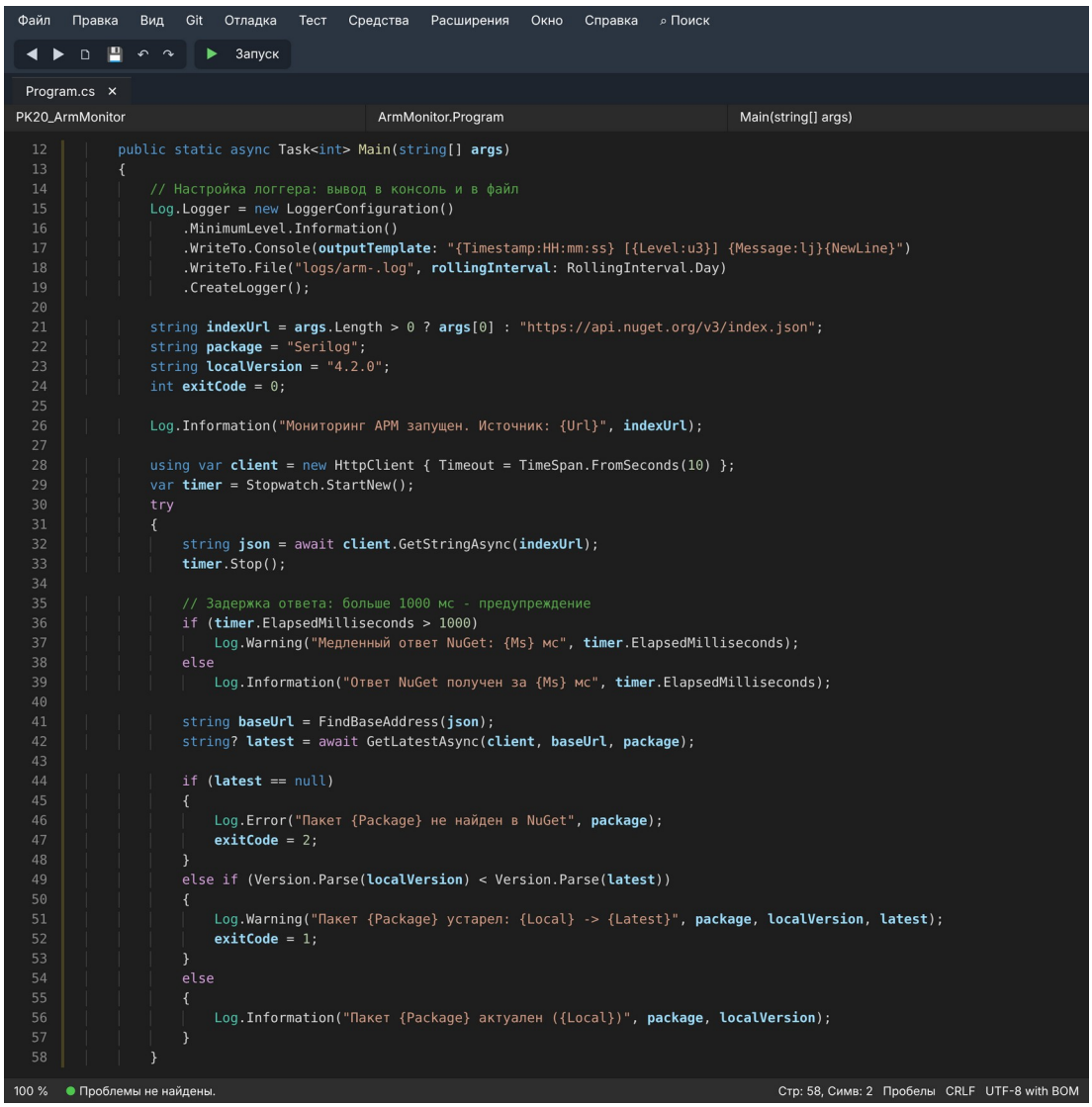


Рисунок 1 – Настройка Serilog и проверка доступности NuGet с замером времени отклика

Вспомогательные методы читают JSON-ответ NuGet и выбирают максимальную стабильную версию пакета.

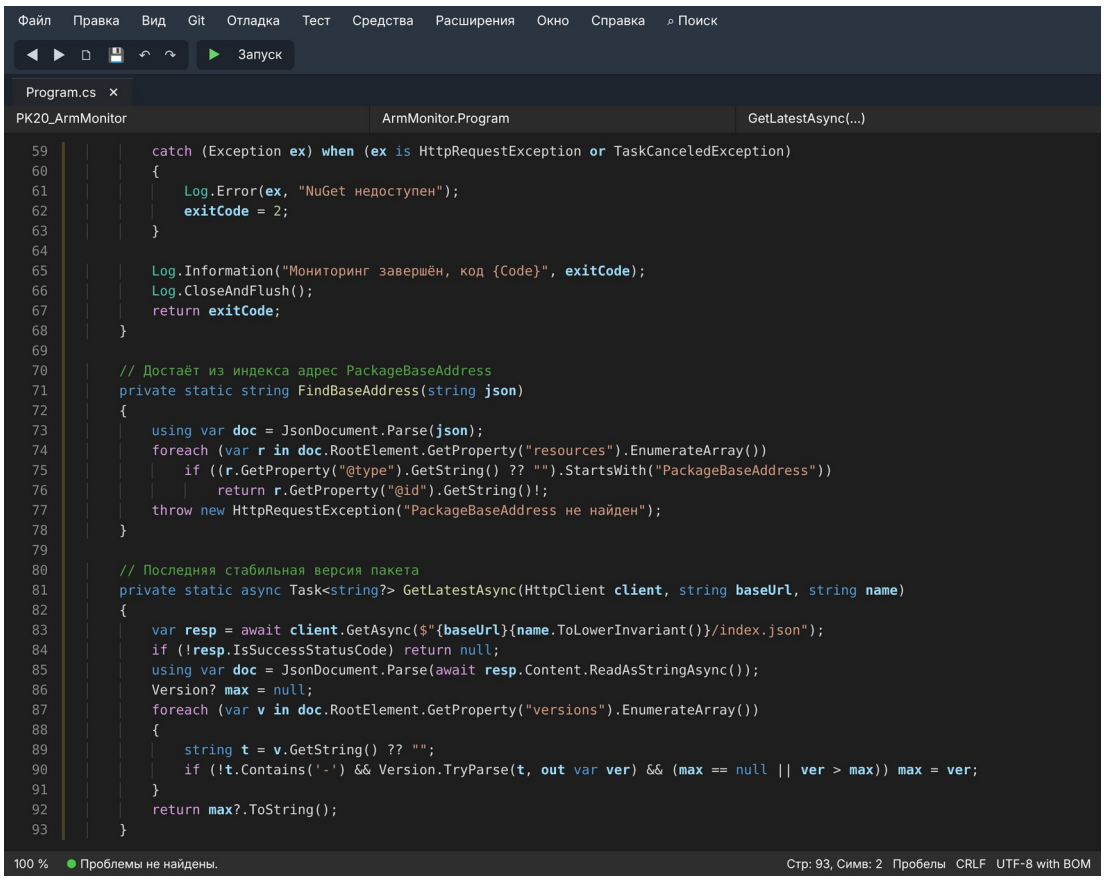


Рисунок 2 – Разбор ответа и поиск последней стабильной версии пакета

2.3 Результат работы программы

При работе программы события выводятся в консоль и сохраняются в файл лога. Ниже приведён результат запуска (рисунок 3). Если сервер недоступен, программа не завершается аварийно: исключение обрабатывается, в журнал пишется запись уровня Error, код завершения равен 2.

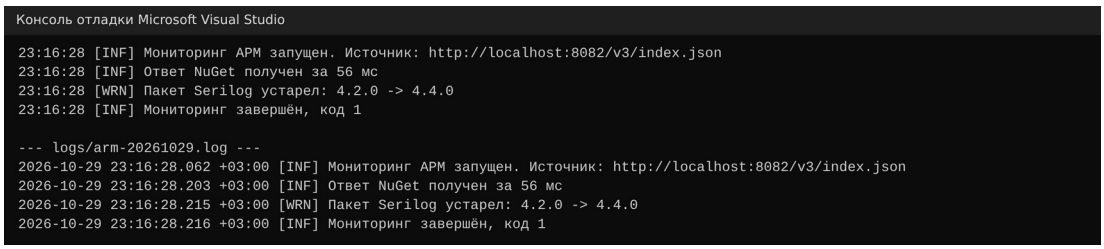


Рисунок 3 – Результат запуска и файл лога

2.4 Вывод

Разработана программа мониторинга для APM дежурного инженера: она проверяет зависимость через NuGet API, замеряет время отклика и пишет структурированный журнал Serilog в консоль и ежедневный файл.

Структурированные логи упрощают поиск проблем: по полям Package, Ms и уровню сообщения можно быстро найти нужные записи, а код завершения позволяет использовать утилиту в автоматических проверках.

3 ПК №21. АРМ БИБЛИОТЕКИ: ИНСПЕКТОР ЗАВИСИМОСТЕЙ И ПРОТОКОЛ SAT

Цель работы: закрепить знания по блокам сопровождения жизненного цикла ПО (CALS) на примере АРМ библиотеки ИС «Умный кампус»: развёртывание и проверка среды, конфигурации и секреты, логирование, приёмочные дымовые тесты и автоматический протокол испытаний.

3.1 Описание задания

Вариант 3: «АРМ библиотеки (Инспектор зависимостей)». Фокус проверки – блоки 1, 3 и 4: проверка целостности локального ПО, проверка доступности пакетов (https://api.nuget.org/v3/index.json [1]) и запись результатов в Markdown-протокол. Журнал событий ведётся через Serilog [4].

Таблица 1 – Дымовые тесты программы

№	Проверка	Критерий успеха
1	Секрет БД	Переменная окружения LIB_DB_PASSWORD задана [6]
2	Целостность файла	SHA-256 файла arm.config совпадает с эталоном [7]
3	Доступность NuGet	Сервер отвечает кодом 2xx

Секретный пароль не хранится в коде: программа читает его из переменной окружения и записывает в лог только факт проверки, но не значение. Итог испытаний сохраняется в файл protocol.md (Docs-as-Code), а код завершения 0 или 1 сообщает об успехе или неудаче.

3.2 Программный код с комментариями

В начале программы настраивается Serilog (консоль и файл logs/smoke-.log). Секрет и адрес NuGet читаются из переменных окружения. Флаг --tamper имитирует порчу файла для проверки сценария с ошибкой.

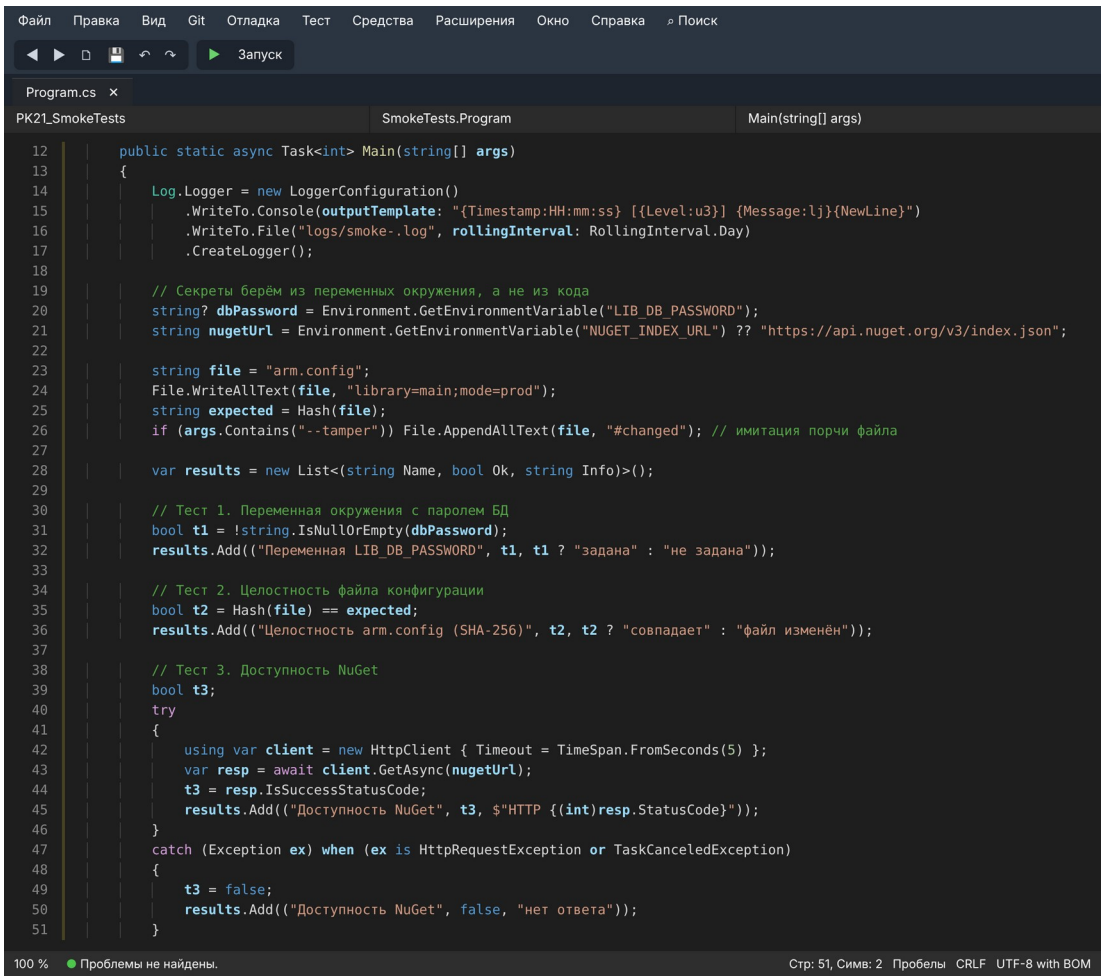


Рисунок 1 – Инициализация Serilog, чтение секретов и выполнение тестов

Результаты тестов записываются в лог и в Markdown-таблицу. Метод Hash считает контрольную сумму файла.

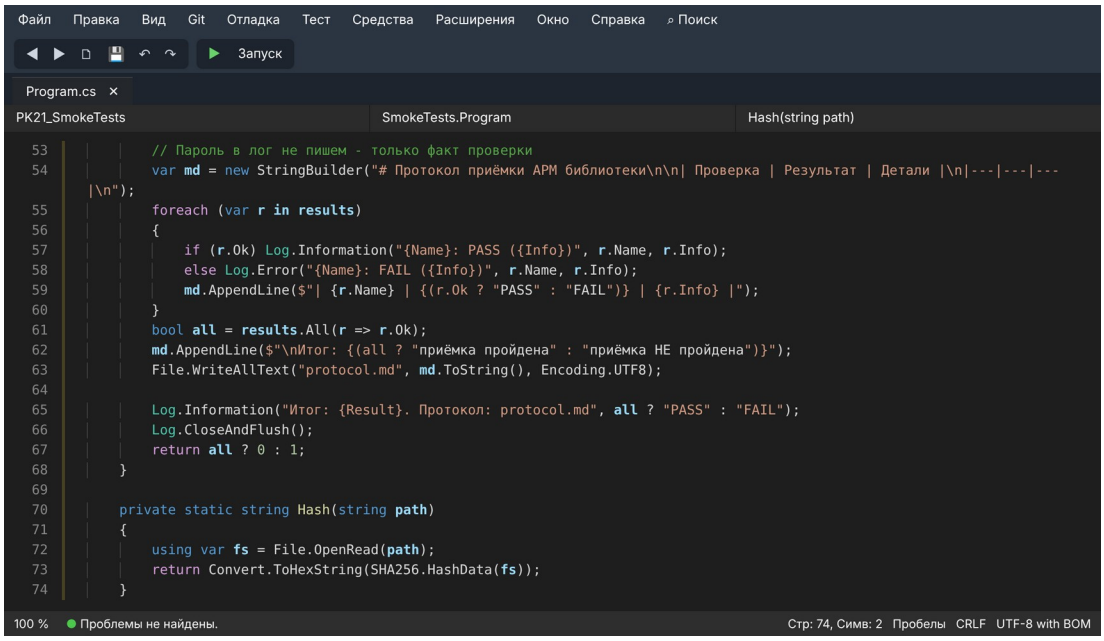


Рисунок 2 – Формирование протокола SAT и вычисление SHA-256

3.3 Результат работы программы

Сценарий 1 (успех): все три теста пройдены, итог «приёмка пройдена» (рисунок 3). Сценарий 2 (ошибка): переменная окружения не задана и файл конфигурации изменён, поэтому приёмка не пройдена, код завершения 1 (рисунок 4).

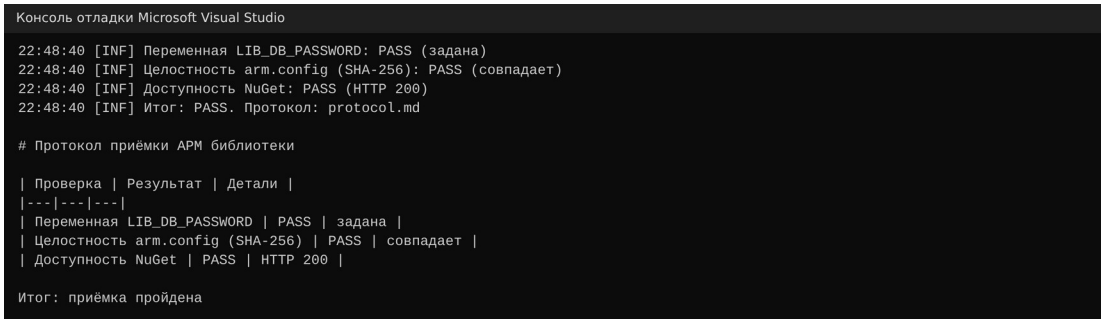


Рисунок 3 – Сценарий успешной приёмки и протокол protocol.md

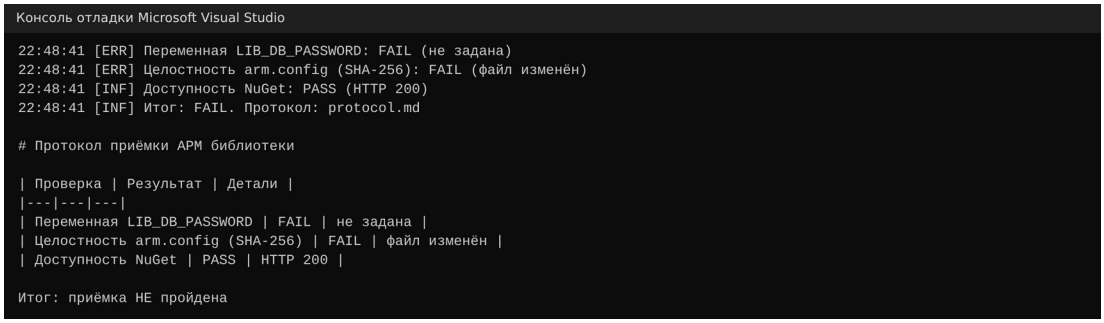


Рисунок 4 – Сценарий с ошибками и протокол protocol.md

3.4 Вывод

Разработана консольная утилита для APM библиотеки, выполняющая комплекс дымовых тестов (секрет окружения, целостность файла по SHA-256, доступность NuGet) и автоматически формирующая протокол приёмки в Markdown. Продемонстрированы сценарии успешного прохождения и ошибки.

Автоматический протокол исключает ручное заполнение документов и показывает, какая именно проверка не пройдена.

4 ПК №22. СИСТЕМА АВТОМАТИЧЕСКОГО ОБНОВЛЕНИЯ (AUTO-UPDATER)

Цель работы: разработать систему автоматического обновления клиентского модуля АРМ проходной ИС «Умный кампус», которая проверяет версию по SemVer, скачивает только изменённые файлы по manifest.json, проверяет их SHA-256 и безопасно заменяет файлы с возможностью отката.

4.1 Описание задания

Клиентский модуль CampusClient.exe версии 2.0.0 работает на АРМ проходной под Windows и .NET 8, сеть нестабильна, поэтому весь дистрибутив скачивать нельзя. Решение SmartCampusSystem состоит из двух проектов: CampusClient (основное приложение) и UpdaterHelper (помощник, заменяющий файлы). Работающая программа не может перезаписать собственный exe-файл, так как операционная система блокирует его, поэтому замену выполняет отдельный процесс.

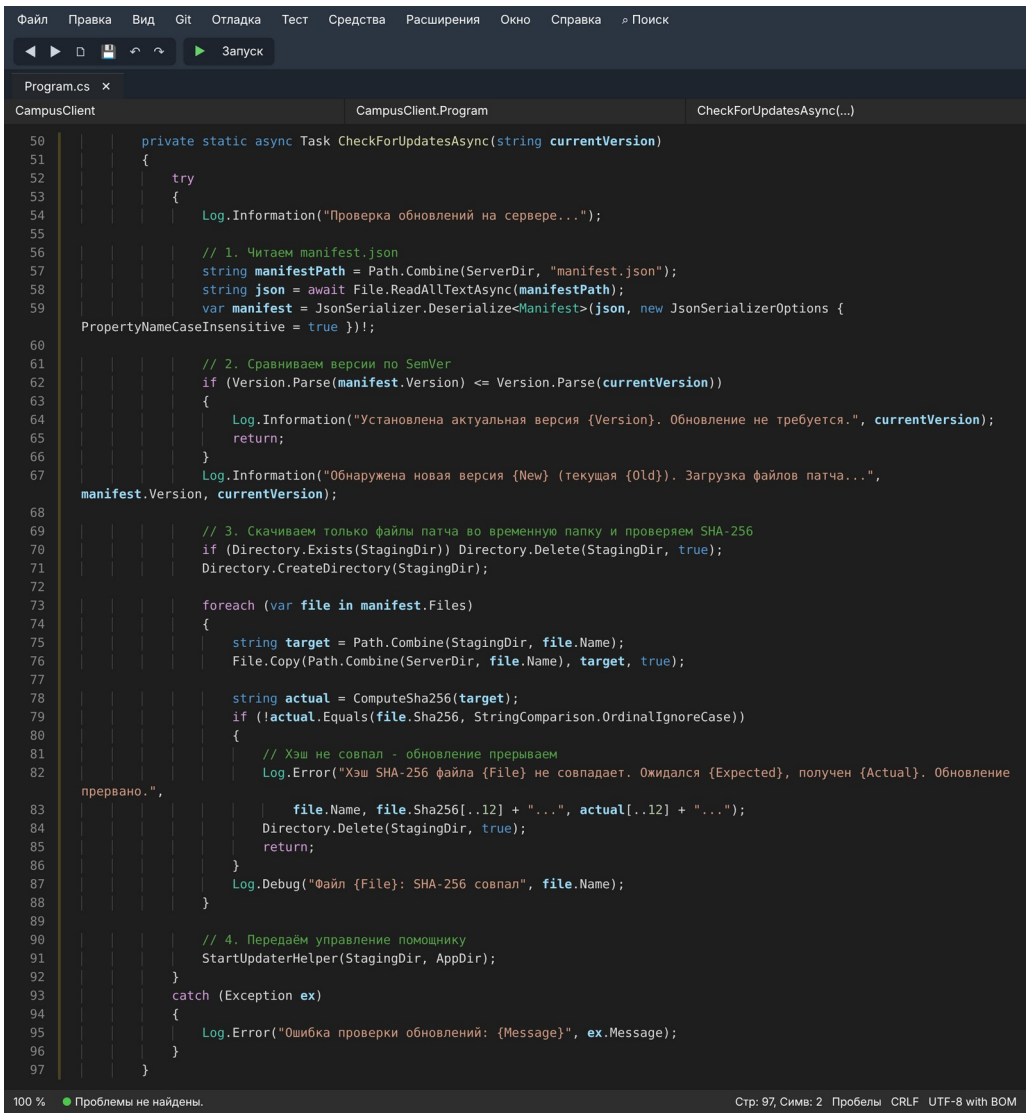
Таблица 1 – Жизненный цикл обновления

Этап	**Что происходит**	**Кто выполняет**
1	Чтение manifest.json, сравнение версий по SemVer [8]	CampusClient
2	Загрузка только файлов патча во временную папку	CampusClient
3	Проверка SHA-256 каждого файла по манифесту [7]	CampusClient
4	Запуск помощника и завершение клиента	CampusClient
5	Резервная копия .bak, замена файлов, откат при ошибке	UpdaterHelper
6	Перезапуск клиента	UpdaterHelper

Дополнительное задание выполнено: при несовпадении хэша обновление прерывается с записью ошибки в лог Serilog [4], а перед перезаписью каждого файла помощник создаёт копию с расширением .bak и при исключении восстанавливает старые файлы.

4.2 Программный код с комментариями

Классы Manifest и PatchFile описывают manifest.json, текущая версия читается из version.txt (по умолчанию 2.0.0). Версии сравниваются через класс Version. Если хэш файла не совпал с манифестом, в лог пишется ошибка и обновление прекращается, иначе запускается UpdaterHelper и клиент завершает работу.




```
        private static async Task CheckForUpdatesAsync(string currentVersion)
        {
            try
            {
                Log.Information("Проверка обновлений на сервере...");

                // 1. Читаем manifest.json
                string manifestPath = Path.Combine(ServerDir, "manifest.json");
                string json = await File.ReadAllTextAsync(manifestPath);
                var manifest = JsonSerializer.Deserialize<Manifest>(json, new JsonSerializerOptions {
PropertyNameCaseInsensitive = true })!;

                // 2. Сравниваем версии по SemVer
                if (Version.Parse(manifest.Version) <= Version.Parse(currentVersion))
                {
                    Log.Information("Установлена актуальная версия {Version}. Обновление не требуется.", currentVersion);
                    return;
                }
                Log.Information("Обнаружена новая версия {New} (текущая {Old}). Загрузка файлов патча...",
manifest.Version, currentVersion);

                // 3. Скачиваем только файлы патча во временную папку и проверяем SHA-256
                if (Directory.Exists(StagingDir)) Directory.Delete(StagingDir, true);
                Directory.CreateDirectory(StagingDir);

                foreach (var file in manifest.Files)
                {
                    string target = Path.Combine(StagingDir, file.Name);
                    File.Copy(Path.Combine(ServerDir, file.Name), target, true);

                    string actual = ComputeSha256(target);
                    if (!actual.Equals(file.Sha256, StringComparison.OrdinalIgnoreCase))
                    {
                        // Хэш не совпал - обновление прерываем
                        Log.Error("Хэш SHA-256 файла {File} не совпадает. Ожидался {Expected}, получен {Actual}. Обновление
прервано.",
                            file.Name, file.Sha256[..12] + "...", actual[..12] + "...");
                        Directory.Delete(StagingDir, true);
                        return;
                    }
                    Log.Debug("Файл {File}: SHA-256 совпал", file.Name);
                }

                // 4. Передаём управление помощнику
                StartUpdaterHelper(StagingDir, AppDir);
            }
            catch (Exception ex)
            {
                Log.Error("Ошибка проверки обновлений: {Message}", ex.Message);
            }
        }
```

Рисунок 1 – CampusClient: проверка версии, загрузка файлов патча и проверка SHA-256 (фрагмент)

Помощник дожидается закрытия клиента по PID, перед перезаписью каждого файла сохраняет копию .bak, а в блоке catch возвращает старые файлы. Флаг simulate_fail.flag имитирует сбой записи для проверки отката. Полный код обоих проектов приведён в архиве проекта SmartCampusSystem.



Рисунок 2 – UpdaterHelper: резервные копии .bak, замена файлов и откат при исключении (фрагмент)

4.3 Результат работы программы и ответы на вопросы

Проверено три сценария. В успешном сценарии (рисунок 3) версия 2.0.0 обновилась до 2.1.0, а повторный запуск показал, что обновление не требуется. При неверном хэше в манифесте (рисунок 4) файлы не заменяются. При сбое

во время замены (рисунок 5) файл CampusClient.exe восстановлен из .bak, клиент запущен снова и со второй попытки обновился.

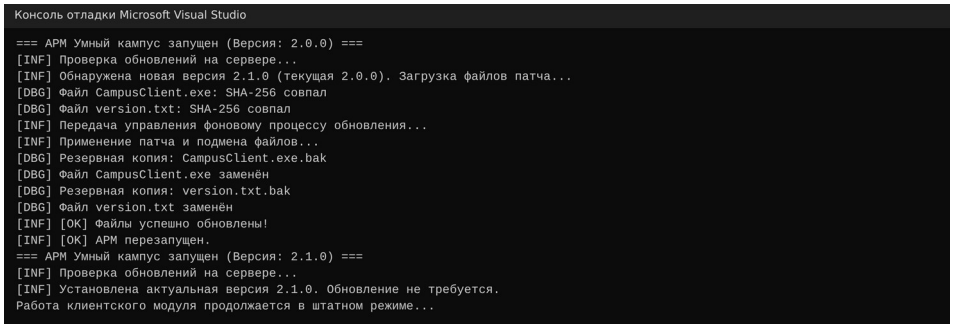


Рисунок 3 – Успешное обновление с версии 2.0.0 до 2.1.0

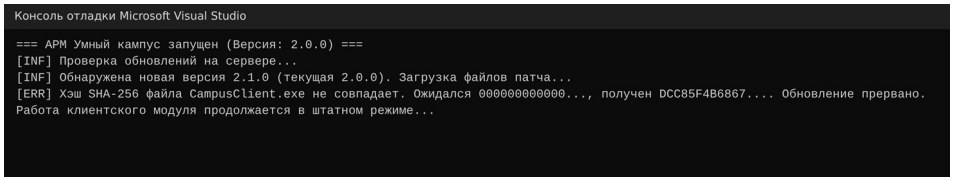


Рисунок 4 – Ошибка проверки SHA-256: обновление прервано

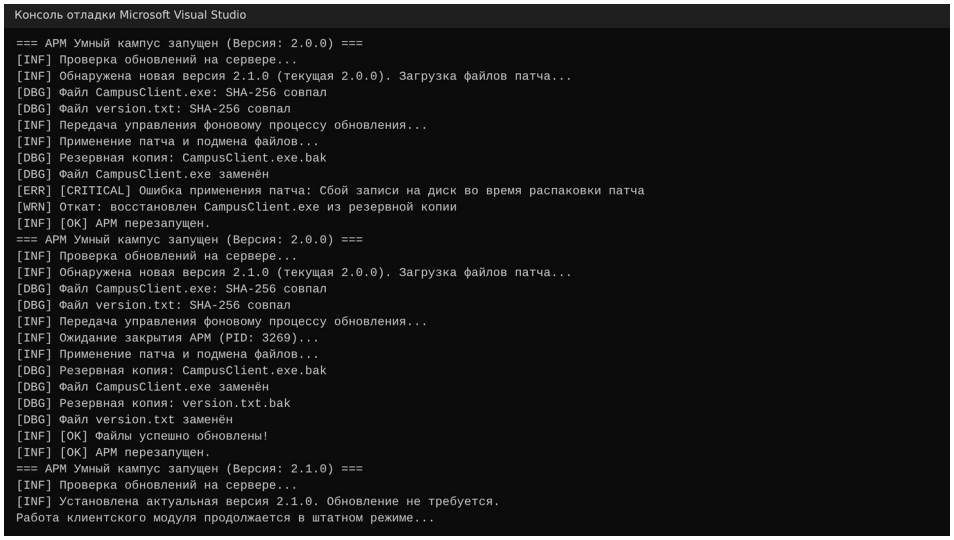


Рисунок 5 – Сбой при замене файлов: откат из .bak

1. Почему CampusClient.exe не может сам перезаписать свой файл?

Пока программа работает, Windows блокирует её exe-файл для записи и удаления. Поэтому клиент завершается, а отдельный процесс UpdaterHelper дожидается его закрытия, заменяет файлы и запускает клиент заново.

2. Чем MINOR отличается от MAJOR?

В SemVer [8] MINOR добавляет совместимые функции, поэтому его можно ставить автоматически. MAJOR может ломать совместимость (формат данных, API), его нельзя накатывать тихо на проходной: нужны проверка и

согласование с администратором, иначе проход студентов может остановиться.

3. Что будет при сбое питания во время работы UpdaterHelper?

Файлы могут остаться частично заменёнными. Защищают резервные копии .bak (в программе откат срабатывает при исключении), хэш-проверка и то, что новые файлы сначала лежат во временной папке. В реальной системе при запуске дополнительно проверяют целостность и при ошибке возвращают .bak, чтобы терминал не завис.

4.4 Вывод

Разработана система автоматического обновления из двух проектов: клиент проверяет версию по SemVer, загружает файлы патча по manifest.json и контролирует их SHA-256, помощник заменяет файлы и перезапускает программу. Реализованы дополнительные механизмы: прерывание обновления при несовпадении хэша с записью в лог Serilog и откат из резервной копии .bak.

Все три сценария (успех, неверный хэш, сбой замены) проверены запуском программы.

5 ПК №23. АВТОНОМНЫЙ ТЕРМИНАЛ СТОЛОВОЙ (EF CORE SQLITE И POLLY)

Цель работы: разработать клиентский модуль АРМ столовой ИС «Умный кампус», который работает при обрыве связи: сохраняет транзакции питания в локальную базу SQLite через Entity Framework Core [9], а при восстановлении сети отправляет их на сервер с помощью политики повторов Polly [10].

5.1 Описание задания

Вариант 3: АРМ столовой – локальный учёт покупок и списания баланса питания по карте. Сущность локального буфера: MealTransaction (StudentId, Amount, Item). Эндпоинт синхронизации: /api/v1/sync/canteen.

Проект CampusOfflineTerminal (Console App, .NET 8) использует пакеты Microsoft.EntityFrameworkCore.Sqlite, Microsoft.EntityFrameworkCore.Design и Polly. Локальная база campus_offline.db содержит две таблицы: CachedCards (кэш карт студентов с балансом) и MealTransactionsQueue (очередь неотправленных транзакций по шаблону Outbox).

Таблица 1 – Модель данных локальной базы

Таблица	**Поля**	**Назначение**
CachedCards	Id, CardCode, StudentId, Balance, IsActive	Проверка карты и баланса офлайн
MealTransactionsQueue	Id, StudentId, Amount, Item, Timestamp, IsSynced	Очередь транзакций для отправки на сервер

5.2 Программный код с комментариями

Контекст LocalCampusDbContext создаёт файл SQLite campus_offline.db. Сущность MealTransaction содержит поля StudentId, Amount, Item и признак IsSynced.

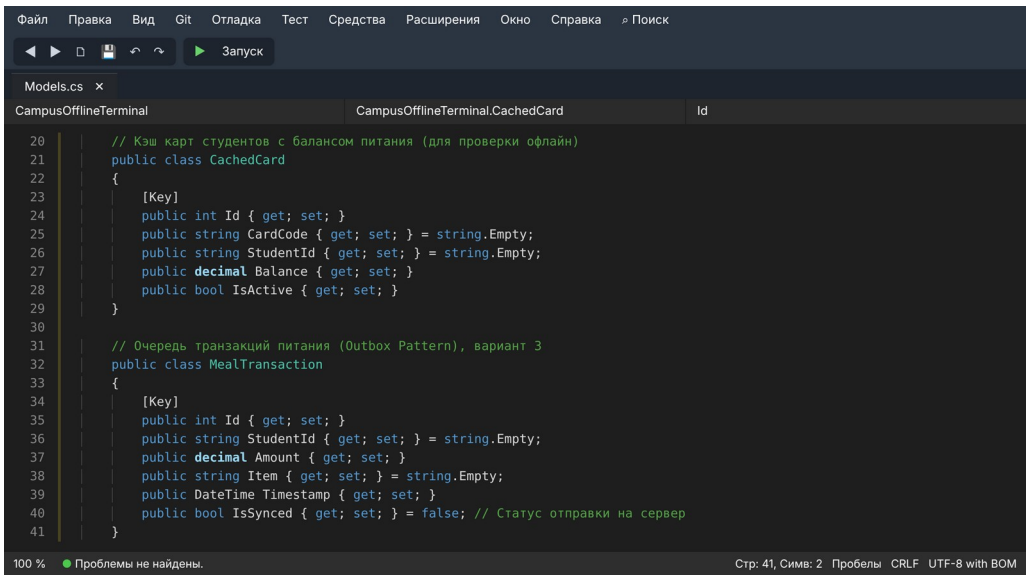


Рисунок 1 – Сущности CachedCard и MealTransaction (фрагмент Models.cs)

Проверка карты и баланса выполняется по локальной базе без обращения к серверу. Списание баланса и запись транзакции сохраняются одним вызовом SaveChanges, поэтому списания без записи в очередь не бывает.

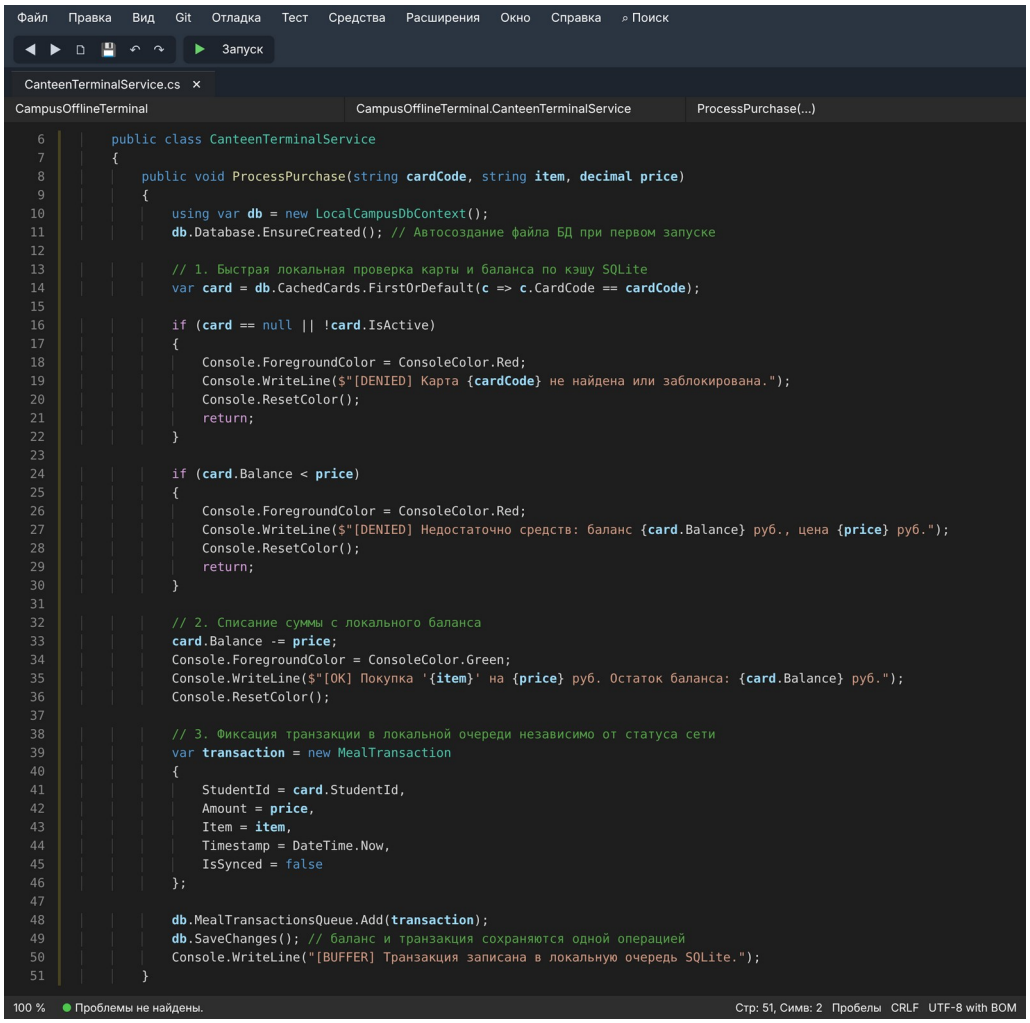


Рисунок 2 – Сервис проверки карты, списания баланса и накопления буфера

Для транзакций, у которых IsSynced = false, выполняется отправка на /api/v1/sync/canteen.

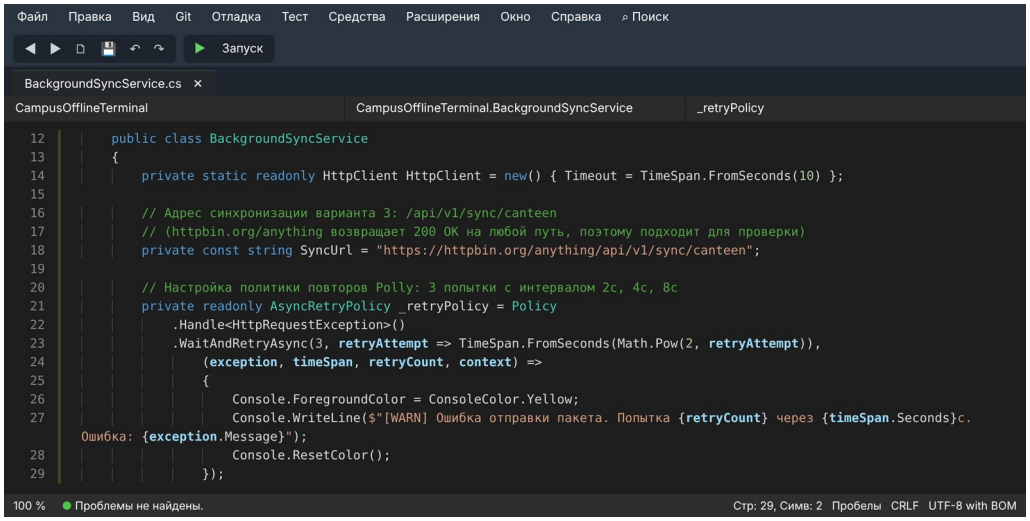


Рисунок 3 – Политика повторов Polly: 3 попытки через 2, 4 и 8 секунд

Если сервер недоступен, после трёх попыток транзакция остаётся в очереди и будет отправлена при следующей синхронизации. Полный листинг проекта CampusOfflineTerminal (файлы Models.cs, CanteenTerminalService.cs, BackgroundSyncService.cs, Program.cs) приведён в архиве проекта.

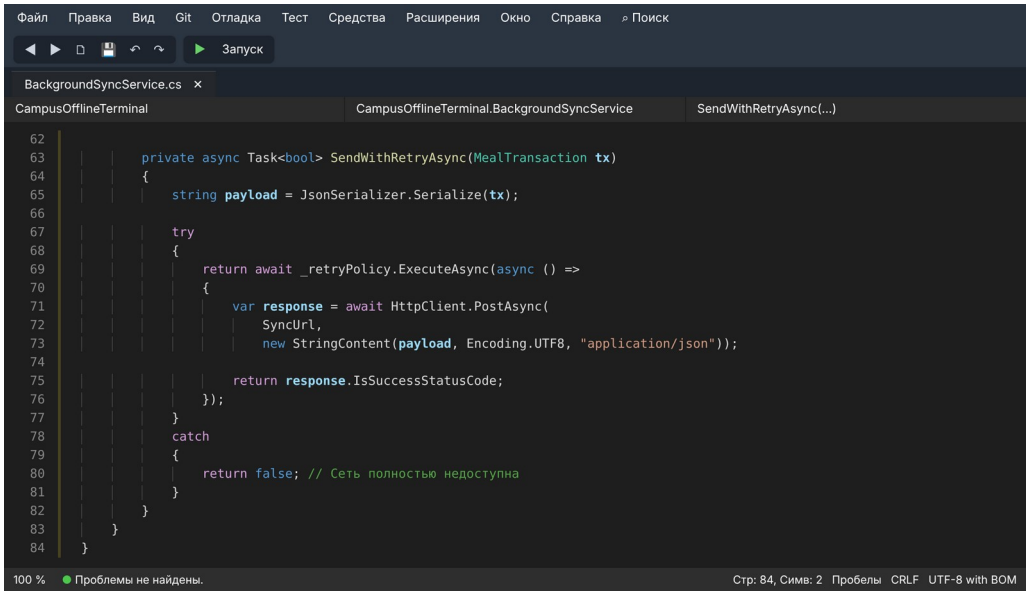


Рисунок 4 – Отправка транзакции на сервер с повторами

5.3 Результат работы программы и ответы на вопросы

Программа запускается в Visual Studio командой Start. Покупки записываются в локальную очередь независимо от состояния сети. Без подключения к сети в консоли появляются сообщения Polly о повторах через 2, 4 и 8 секунд и запись «SYNC FAIL» (транзакции остаются в буфере). При

работающей сети транзакции отправляются, и появляются сообщения «SYNC OK».

1. Почему критичен подход Offline-First с SQLite?

Касса столовой должна обслуживать студентов даже без связи с сервером. Локальная база SQLite отвечает за миллисекунды, не зависит от сети и не создаёт очередей при считывании карт.

2. Как сохраняются данные при обрыве связи и почему ничего не теряется?

Каждая транзакция сначала записывается в таблицу MealTransactionsQueue с признаком IsSynced = false. Признак меняется на true только после ответа сервера, поэтому при сбое запись остаётся в очереди и отправляется позже.

3. Почему используется экспоненциальная задержка, а не бесконечный цикл запросов?

Постоянные запросы нагружают сеть и уже перегруженный сервер. Интервалы 2, 4, 8 секунд дают серверу время восстановиться [11], а число попыток ограничено, чтобы программа не зависала.

5.4 Вывод

Разработан модуль АРМ столовой по принципу Offline-First: карты и баланс проверяются по локальной базе SQLite через EF Core, транзакции питания копятся в очереди (Outbox), а фоновая синхронизация с политикой повторов Polly отправляет их на /api/v1/sync/canteen.

Подход гарантирует, что ни одна покупка не будет потеряна при сбое сети, и не нагружает сервер лишними запросами.

6 ПК №24. ИНФОРМАЦИОННАЯ БЕЗОПАСНОСТЬ АРМ: BCRYPT, X.509 И АУДИТ SERILOG

Цель работы: реализовать защищённый модуль аутентификации и аудита для АРМ библиотеки ИС «Умный кампус»: хэширование паролей через BCrypt [12], проверка срока действия клиентских сертификатов X.509 [13] и структурированный журнал безопасности Serilog [4].

6.1 Описание задания

Вариант 3: «АРМ библиотеки: проверка учётных данных библиотекаря и аудит запросов к закрытым фондам». Ключевой компонент безопасности: BCrypt + Serilog Audit. Целевая роль: Operator.

Проект CampusSecurityTerminal (Console App, .NET 8) использует пакеты BCrypt.Net-Next, Serilog и Serilog.Sinks.File. Класс SecurityService хэширует пароль, проверяет его при входе, проверяет сертификат и пишет события в изолированный журнал security_audit.log. Для варианта 3 добавлен метод AuditFundRequest, который фиксирует каждый запрос библиотекаря к закрытому фонду.

Таблица 1 – Методы класса SecurityService

Метод	**Назначение**	**Запись в журнале**
HashPassword	Хэш пароля BCrypt при регистрации	Information
VerifyPassword	Проверка пароля при входе	Information / Warning
AuditFundRequest	Аудит доступа к закрытому фонду	Information / Warning
VerifyClientCertificate	Срок действия и цепочка сертификата X.509	Information / Warning / Error

6.2 Программный код с комментариями

Журнал безопасности настраивается отдельно в файле Logs\ security_audit.log. Пароль никогда не пишется в лог: фиксируется только результат проверки.

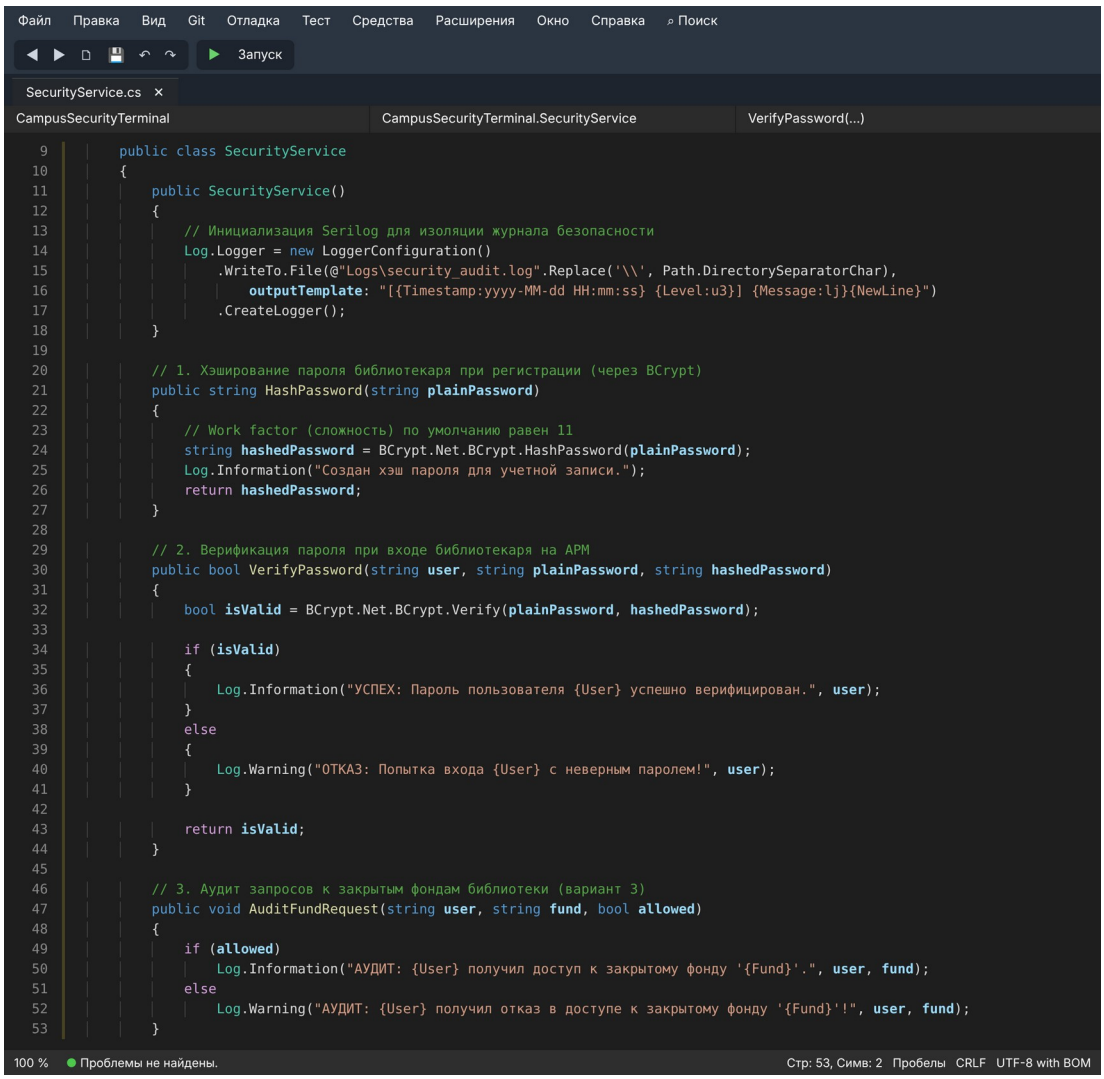


Рисунок 1 – SecurityService: Serilog, хэширование и проверка пароля, аудит запросов к фондам

Метод проверяет наличие файла, срок действия (NotBefore и NotAfter) и цепочку доверия.

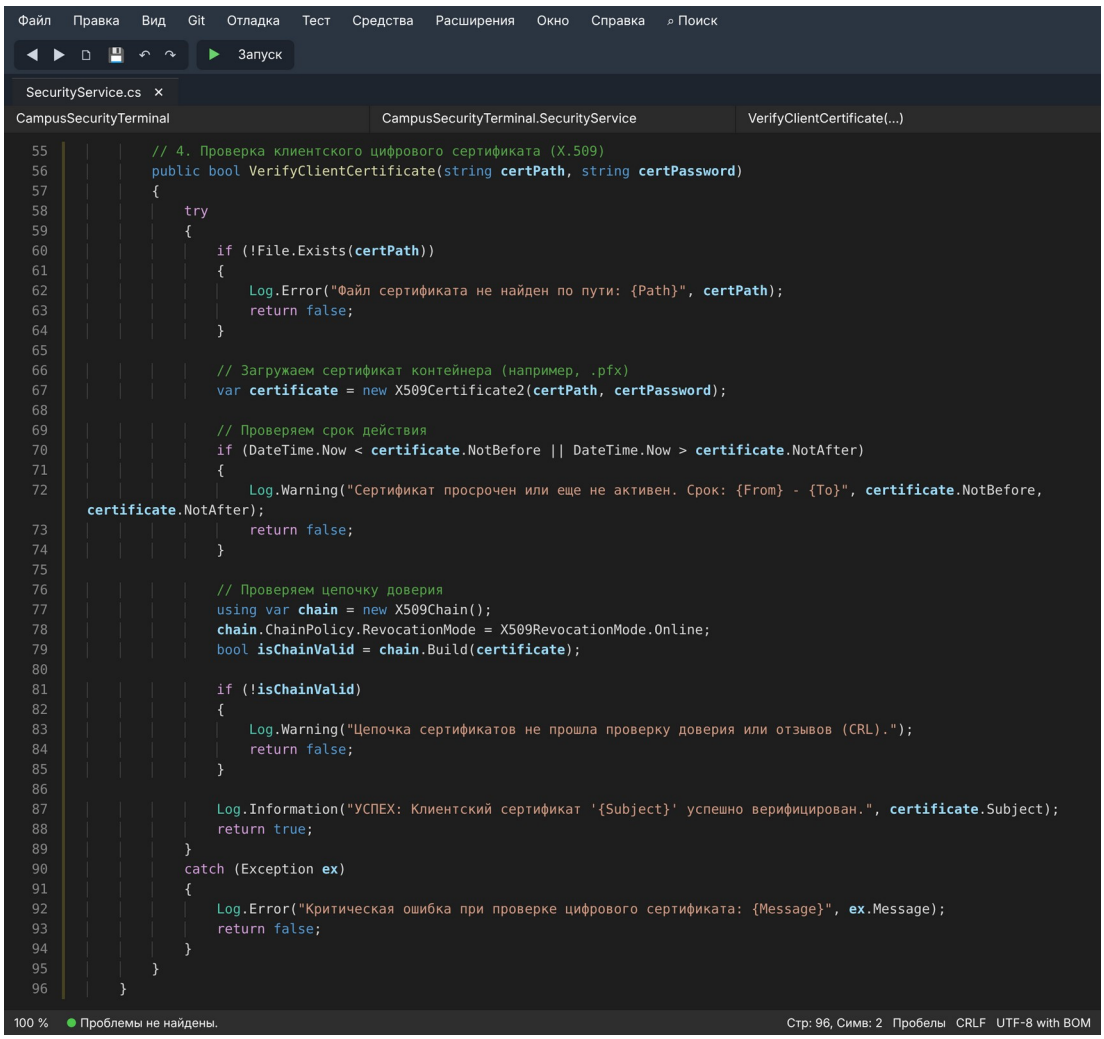


Рисунок 2 – SecurityService: проверка сертификата X.509

Сценарии: верный и неверный пароль, аудит запросов, проверка отсутствующего и просроченного сертификата. Просроченный сертификат создаётся в самой программе для проверки.

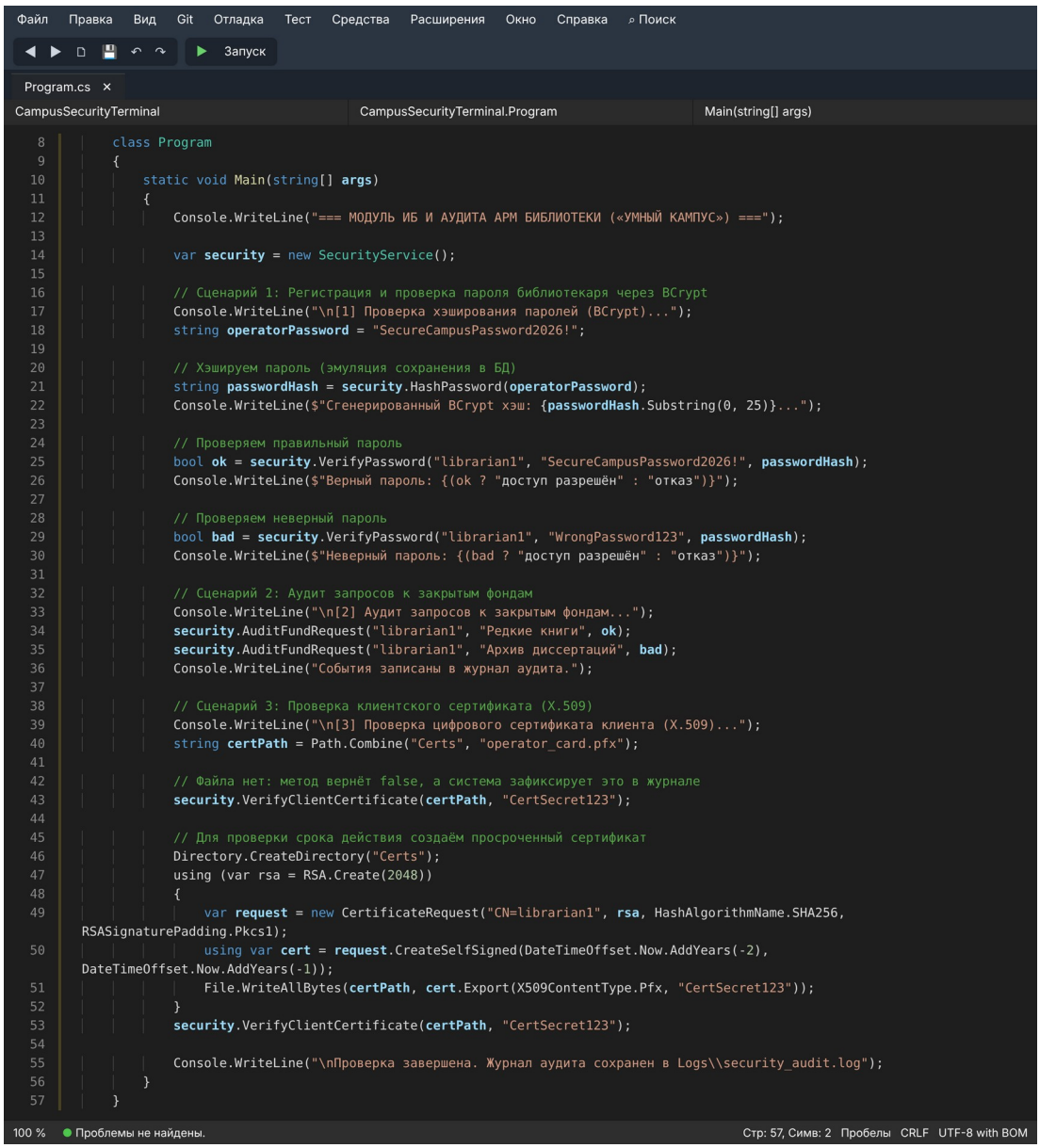


Рисунок 3 – Program.cs: сценарии работы модуля

6.3 Результат работы программы и ответы на вопросы

Результат работы программы показан на рисунке 4, содержимое журнала аудита – на рисунке 5. В журнале зафиксированы успешная проверка пароля, отказ при неверном пароле, запросы к закрытым фондам, отсутствие файла сертификата и просроченный сертификат.

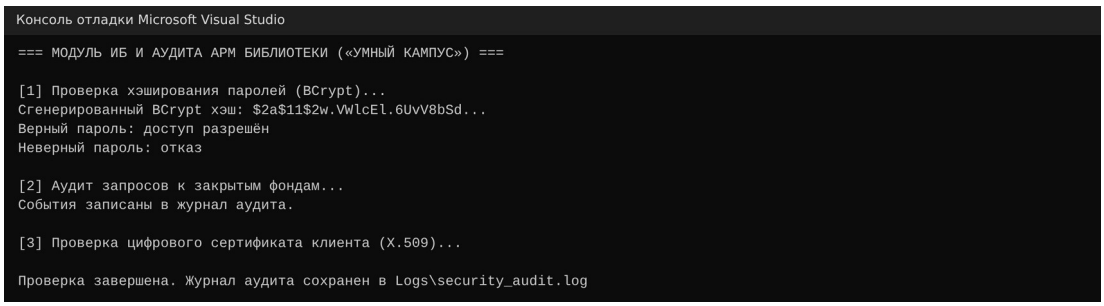


Рисунок 4 – Результат работы программы

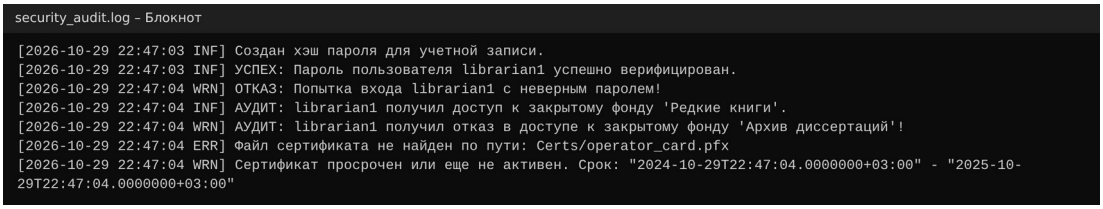


Рисунок 5 – Журнал аудита security_audit.log

1. Почему нельзя хранить пароли через MD5 или SHA-256, а BCrypt – стандарт?

MD5 и SHA-256 вычисляются очень быстро, поэтому злоумышленник перебирает миллиарды вариантов в секунду. BCrypt специально медленный, использует соль и настраиваемый фактор сложности (work factor), который можно увеличивать с ростом мощности компьютеров [14].

2. Какие два параметра сертификата проверяются в первую очередь?

Срок действия (даты NotBefore и NotAfter) и подлинность: цепочка доверия до доверенного центра сертификации с проверкой отзыва (CRL).

3. Зачем отдельный журнал событий безопасности?

События безопасности нужно хранить изолированно: к ним ограничивают доступ, их не затирают общими логами, в них проще искать попытки взлома и ошибки сертификатов, и они служат доказательством при расследовании инцидента.

6.4 Вывод

Реализован модуль безопасности АРМ библиотеки: пароли библиотекаря хранятся и проверяются через BCrypt, сертификаты проверяются по сроку действия и цепочке доверия, а все события (вход, отказы, запросы к закрытым фондам, ошибки сертификата) пишутся в отдельный журнал Serilog.

Программа запущена, журнал аудита подтверждает корректную работу всех сценариев.

ЗАКЛЮЧЕНИЕ

В сводном отчёте представлены шесть работ, выполненные как отдельные программы на C# (.NET 8): проверка NuGet-зависимостей, мониторинг с логом Serilog, дымовые тесты с протоколом SAT, автоматическое обновление с проверкой SHA-256 и откатом, автономный терминал на SQLite и Polly, модуль безопасности на BCrypt и X.509.

Вместе эти решения образуют цикл сопровождения ПО: перед внедрением проверяются зависимости и окружение, при внедрении формируется протокол приёмки, в эксплуатации ПО обновляется безопасно и работает без сети, а события безопасности фиксируются в журнале аудита.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Microsoft Learn. NuGet API overview [Электронный ресурс]. – URL: https://learn.microsoft.com/en-us/nuget/api/overview (дата обращения: 29.10.2026).

2. Microsoft Learn. HttpClient Class [Электронный ресурс]. – URL: https://learn.microsoft.com/en-us/dotnet/api/system.net.http.httpclient (дата обращения: 29.10.2026).

3. Microsoft Learn. Package Base Address, NuGet API [Электронный ресурс]. – URL: https://learn.microsoft.com/en-us/nuget/api/package-base-address-resource (дата обращения: 29.10.2026).

4. Serilog. Structured logging for .NET [Электронный ресурс]. – URL: https://serilog.net (дата обращения: 29.10.2026).

5. Serilog.Sinks.File. Документация проекта [Электронный ресурс]. – URL: https://github.com/serilog/serilog-sinks-file (дата обращения: 29.10.2026).

6. Microsoft Learn. Environment.GetEnvironmentVariable Method [Электронный ресурс]. – URL: https://learn.microsoft.com/en-us/dotnet/api/system.environment.getenvironmentvariable (дата обращения: 29.10.2026).

7. Microsoft Learn. SHA256 Class [Электронный ресурс]. – URL: https://learn.microsoft.com/en-us/dotnet/api/system.security.cryptography.sha256 (дата обращения: 29.10.2026).

8. Semantic Versioning 2.0.0 [Электронный ресурс]. – URL: https://semver.org/lang/ru/ (дата обращения: 29.10.2026).

9. Microsoft Learn. SQLite Database Provider – EF Core [Электронный ресурс]. – URL: https://learn.microsoft.com/en-us/ef/core/providers/sqlite/ (дата обращения: 29.10.2026).

10. Polly Documentation. Retry resilience strategy [Электронный ресурс]. – URL: https://www.pollydocs.org/strategies/retry.html (дата обращения: 29.10.2026).

11. Microsoft Learn. Retry pattern [Электронный ресурс]. – URL: https://learn.microsoft.com/en-us/azure/architecture/patterns/retry (дата обращения: 29.10.2026).

12. BCrypt.Net-Next. Документация проекта [Электронный ресурс]. – URL: https://github.com/BcryptNet/bcrypt.net (дата обращения: 29.10.2026).

13. Microsoft Learn. X509Certificate2 Class [Электронный ресурс]. – URL: https://learn.microsoft.com/en-us/dotnet/api/system.security.cryptography.x509certificates.x509certificate2 (дата обращения: 29.10.2026).

14. OWASP. Password Storage Cheat Sheet [Электронный ресурс]. – URL: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (дата обращения: 29.10.2026).